Job Tracker Backend Documentation

Covers authentication, database layer, REST API, middleware, background jobs, and email delivery.

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Technology Stack
4. Project Structure
5. Data Model
6. Authentication
7. Middleware
8. API Reference
9. Background Jobs
10. Email System
11. Configuration and Environment

---

1) Executive Summary

Job Tracker is a **Node.js + Express** REST API that powers a full-stack job application management system. The server is a standalone process decoupled from the React frontend, communicating exclusively over HTTP.

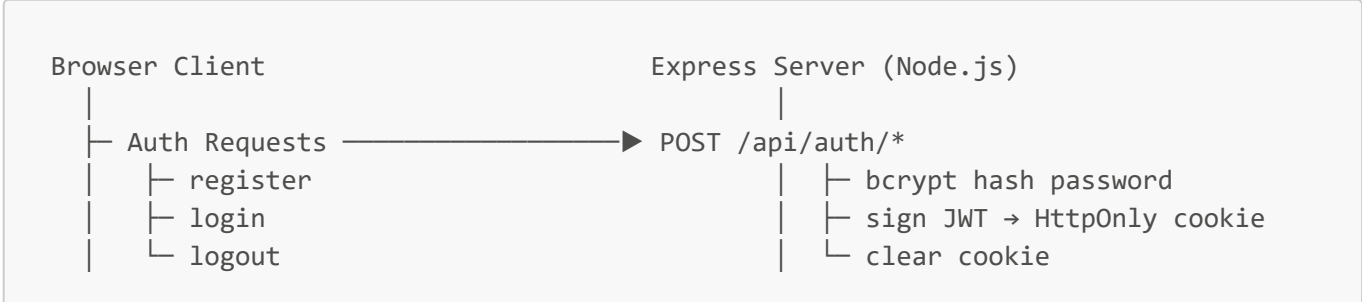
At a high level:

- **Express 5** handles routing, middleware, and HTTP request/response lifecycle.
- **MongoDB + Mongoose** persists user accounts and job application records.
- **JWT** tokens issued on login are stored in **HttpOnly cookies**, providing secure, stateless session management.
- **bcrypt** hashes all user passwords before storage; plaintext passwords are never persisted.
- **node-cron** schedules a daily job at 9 AM that scans for due follow-up dates and sends email reminders.
- **Nodemailer** delivers reminder emails through Gmail SMTP.
- **Helmet**, **CORS**, and **express-rate-limit** harden the API against common web threats.

---

2) Architecture Overview

2.1 System Context



```
  │                                          │
  ├─ Application CRUD ──────────────────▶ /api/applications/*
  │   (JWT cookie sent automatically)        │   ├─ protect middleware validates
token
  │   ├─ GET    /                           │   ├─ ownership check on mutating
ops
  │   ├─ POST   /                           │   └─ Mongoose CRUD
  │   ├─ PUT    /:id                        │
  │   └─ DELETE /:id                         │
  │                                          ▼
  └─ TradingView / Client UI          MongoDB Atlas
                                        ├─ users        (email, hashed password)
                                        └─ applications (all job application
fields)

node-cron (internal scheduler)
  └─ 0 9 * * *  ──────────────────────▶ sendReminders()
                                          ├─ query applications where followUpDate
<= today
                                          │  AND reminderSent = false
                                          ├─ send email via Nodemailer (Gmail
SMTP)
                                          └─ mark reminderSent = true
```

2.2 Runtime Model

The server is a single long-running Node.js process. There is no separate job worker — the cron scheduler (`node-cron`) runs inside the same process and is registered at startup via `server.js`.

Key design decisions:

- **Stateless JWT auth** — No session store is needed. The JWT payload carries `userId`; the `protect` middleware verifies the signature and loads the user from MongoDB on each request.
- **Dual token source** — The `protect` middleware accepts a token from either the `Authorization: Bearer` header or the `token` HttpOnly cookie, supporting both browser clients (cookie) and API clients (header).
- **Resource ownership enforcement** — `updateApplication` and `deleteApplication` perform an explicit ownership check (`application.user === req.user._id`) after fetching the document, preventing users from modifying each other's data.
- **Rate limiting scoped to auth routes** — The limiter is applied only to `/api/auth` to throttle brute-force login and registration attempts without impacting application data endpoints.

---

3) Technology Stack

3.1 Core Platform

Package	Version	Role
Node.js	LTS	Runtime

Package	Version	Role
Express	5.x	HTTP framework — routing, middleware, request handling

3.2 Data Layer

Package	Version	Role
MongoDB (Atlas)	—	Primary database
Mongoose	9.x	ODM — schema definition, validation, querying

3.3 Authentication and Security

Package	Version	Role
jsonwebtoken	9.x	JWT signing and verification
bcrypt	6.x	Password hashing (10 salt rounds)
cookie-parser	1.x	Parses `token` cookie from incoming requests
helmet	8.x	Sets secure HTTP response headers
cors	2.x	Cross-origin request policy with allowlist
express-rate-limit	8.x	Rate limiting for auth endpoints

3.4 Background Jobs and Email

Package	Version	Role
node-cron	4.x	In-process cron scheduler
nodemailer	7.x	SMTP email transport via Gmail

3.5 Utility

Package	Role
dotenv	Environment variable loading from `.env`
nodemon	Dev-only — auto-restarts server on file changes

---

4) Project Structure

```
server/
  server.js                    # Entry point: loads env, connects DB, starts
cron, binds port
  app.js                       # Express app: global middleware, route mounting
  config/
    db.js                      # Mongoose connection function
```

```
    mailer.js                  # Shared Nodemailer transporter instance
  controllers/
    auth.controller.js         # register, login, logout handlers
    application.controller.js  # createApplication, getApplications,
updateApplication, deleteApplication
  middleware/
    auth.middleware.js         # protect — JWT verification via header or cookie
    validateObjectId.js        # Guards /:id routes against malformed MongoDB
ObjectIds
  models/
    User.js                    # User schema: email, hashed password
    Application.js             # Application schema: all job tracking fields
  routes/
    auth.routes.js             # POST /register, /login, /logout
    application.routes.js      # CRUD routes — all protected
  jobs/
    reminder.job.js            # Cron job: daily follow-up email reminders
```

---

5) Data Model

5.1 `users` Collection

Managed by Mongoose via `User.js`. Stores registered user credentials.

Field	Type	Constraints	Notes
`email`	String	Required, unique, indexed	Lowercased and trimmed before save
`password`	String	Required	bcrypt hash — plaintext is never stored
`createdAt`	Date	Auto	Mongoose timestamps
`updatedAt`	Date	Auto	Mongoose timestamps

5.2 `applications` Collection

Managed by Mongoose via `Application.js`. Stores all job application records for all users.

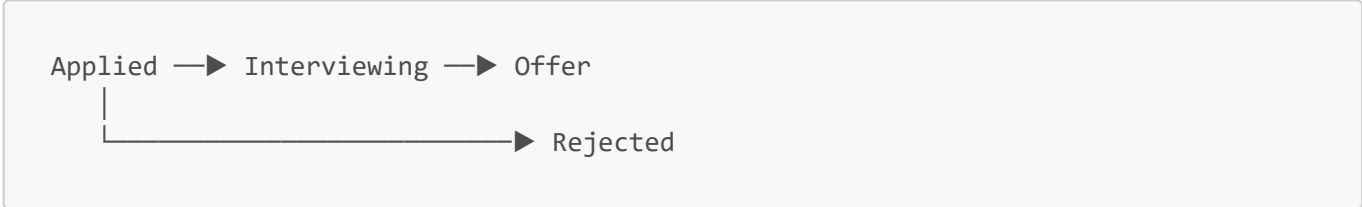
Field	Type	Constraints	Notes
`user`	ObjectId	Required, ref: `User`	Owner of the application
`company`	String	Required, trimmed	Company name
`role`	String	Required, trimmed	Job title / role applied for
`status`	String	Enum, default: `"Applied"`	One of: `Applied`, `Interviewing`, `Offer`, `Rejected`
`appliedDate`	Date	Default: `Date.now`	Date the application was submitted
`followUpDate`	Date	Optional	Date on which a follow-up reminder should be sent

Field	Type	Constraints	Notes
`notes`	String	Optional	Free-text notes about the application
`jobLink`	String	Optional	URL to the job posting
`reminderSent`	Boolean	Default: `false`	Flipped to `true` after the cron job sends an email reminder
`createdAt`	Date	Auto	Mongoose timestamps
`updatedAt`	Date	Auto	Mongoose timestamps

Indexes:

- `user` — implicitly indexed via the ObjectId ref; all queries filter by `user: req.user._id`
- `{ followUpDate, reminderSent }` — queried together by the reminder cron job

5.3 Application Status Lifecycle



Status transitions are not enforced server-side — the client may set any valid enum value freely.

---

6) Authentication

File: server/controllers/auth.controller.js

6.1 Token Strategy

Authentication uses **JWT** tokens stored in **HttpOnly cookies**. The token is signed with `JWT_SECRET` and expires after **7 days**. Because the cookie is HttpOnly, it is inaccessible to JavaScript running in the browser, preventing XSS-based token theft.

```
register / login
  └─ generateToken(userId)
       └─ jwt.sign({ userId }, JWT_SECRET, { expiresIn: '7d' })
  └─ setAuthCookie(res, token)
       └─ res.cookie('token', token, {
             httpOnly: true,
             secure: true (production),
             sameSite: 'strict' (production) | 'lax' (development),
             maxAge: 7 days
           })
```

6.2 Auth Settings

Setting	Value	Notes
Token type	JWT	Signed with `HS256` (jsonwebtoken default)
Token lifetime	7 days	Controlled by `expiresIn: '7d'` and `maxAge` cookie option
Password hashing	bcrypt, 10 salt rounds	Applied at registration; compared at login
Cookie name	`token`	HttpOnly, not accessible to client-side JS
`secure` flag	`true` in production	Enforces HTTPS-only cookie transmission
`sameSite`	`strict` in production, `lax` in development	Mitigates CSRF

6.3 Controller Functions

Function	Endpoint	Description
`register`	`POST /api/auth/register`	Validates input, checks for duplicate email, hashes password with bcrypt, creates user, signs JWT, sets auth cookie
`login`	`POST /api/auth/login`	Looks up user by email, compares password hash with bcrypt, signs JWT, sets auth cookie
`logout`	`POST /api/auth/logout`	Clears the `token` cookie with matching options

6.4 Registration Flow

```
POST /api/auth/register
  │
  ├─ Validate: email + password present
  ├─ Check: User.findOne({ email }) → 400 if exists
  ├─ bcrypt.hash(password, 10)
  ├─ User.create({ email, hashedPassword })
  ├─ generateToken(user._id)
  ├─ setAuthCookie(res, token)
  └─ 201 { message: "Registered" }
```

6.5 Login Flow

```
POST /api/auth/login
  │
  ├─ User.findOne({ email }) → 400 if not found
  ├─ bcrypt.compare(password, user.password) → 400 if no match
  ├─ generateToken(user._id)
```

```
    ├─ setAuthCookie(res, token)
    └─ 200 { message: "Logged in" }
```

---

7) Middleware

7.1 `protect` — JWT Auth Guard

File: server/middleware/auth.middleware.js

Applied as a router-level middleware to all `/api/applications` routes. Blocks unauthenticated requests before they reach any controller.

Token resolution order:

1. Check `Authorization` header for `Bearer <token>` — used by non-browser API clients.
2. Fall back to `req.cookies.token` — used by the browser client.
3. If neither is present, respond `401 No token provided`.

After verifying the JWT signature, the middleware fetches the user from MongoDB (`User.findById`) and attaches it to `req.user` (excluding the password field). If the token is valid but the user no longer exists, the request is rejected with `401 Not authorized`.

Condition	Response
Valid token, user found	Calls `next()` — `req.user` is populated
Valid token, user not in DB	`401 Not authorized`
Invalid or expired token	`401 Not authorized`
No token at all	`401 No token provided`

7.2 `validateObjectId` — Route Parameter Guard

File: server/middleware/validateObjectId.js

Applied before `updateApplication` and `deleteApplication`. Checks that `req.params.id` is a valid MongoDB ObjectId using `mongoose.Types.ObjectId.isValid()`. Returns `400 Invalid ID` immediately if the check fails, preventing Mongoose from throwing a `CastError` inside the controller.

7.3 `authLimiter` — Rate Limiter

File: server/app.js

Scoped to all `/api/auth` routes. Limits each IP to **20 requests per 15-minute window** using `express-rate-limit`. Exceeding the limit returns a `429 Too Many Requests` response with standard `RateLimit-*` headers.

Setting	Value
Window	15 minutes

Setting	Value
Max requests	20 per IP
Headers	Standard (`RateLimit-*`), legacy headers disabled

7.4 Global Middleware (app.js)

Middleware	Purpose
`helmet()`	Sets 15+ secure HTTP headers (CSP, HSTS, X-Frame-Options, etc.)
`cors({ origin, credentials })`	Restricts origins to the `CORS_ORIGIN` allowlist; allows cookies via `credentials: true`
`cookieParser()`	Populates `req.cookies` so the auth middleware can read the `token` cookie
`express.json()`	Parses JSON request bodies into `req.body`

8) API Reference

Base URL: `/api`

All `/api/applications` routes require a valid `token` cookie or `Authorization: Bearer <token>` header.

8.1 Auth Routes

Rate-limited: 20 requests per 15 minutes per IP.

`POST /api/auth/register`

Creates a new user account and sets an auth cookie.

Request body:

```
{
  "email": "user@example.com",
  "password": "securepassword"
}
```

Responses:

Status	Body	Condition
`201`	`{ "message": "Registered" }`	User created successfully
`400`	`{ "message": "All fields are required" }`	Missing email or password
`400`	`{ "message": "User already exists" }`	Email already in use

Status	Body	Condition
500	{ "message": "Server error" }	Unexpected server failure

`POST /api/auth/login`

Authenticates an existing user and sets an auth cookie.

Request body:

```
{
  "email": "user@example.com",
  "password": "securepassword"
}
```

Responses:

Status	Body	Condition
200	{ "message": "Logged in" }	Auth cookie set
400	{ "message": "Invalid credentials" }	Email not found or password mismatch
500	{ "message": "Server error" }	Unexpected server failure

`POST /api/auth/logout`

Clears the auth cookie.

Responses:

Status	Body
200	{ "message": "Logged out" }

8.2 Application Routes

All routes require authentication. The `protect` middleware runs on the entire router.

`GET /api/applications`

Returns all job applications belonging to the authenticated user, sorted by creation date descending.

Responses:

Status	Body
200	Array of application objects

Status	Body
401	{ "message": "Not authorized" }
500	{ "message": "Server error" }

Example response item:

```
{
  "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
  "user": "64f1a2b3c4d5e6f7a8b9c0d0",
  "company": "Acme Corp",
  "role": "Software Engineer",
  "status": "Interviewing",
  "appliedDate": "2025-03-01T00:00:00.000Z",
  "followUpDate": "2025-03-10T00:00:00.000Z",
  "notes": "Spoke with recruiter on 3/5",
  "jobLink": "https://acme.com/jobs/123",
  "reminderSent": false,
  "createdAt": "2025-03-01T12:00:00.000Z",
  "updatedAt": "2025-03-05T09:00:00.000Z"
}
```

`POST /api/applications`

Creates a new job application for the authenticated user.

Request body (all optional fields may be omitted):

```
{
  "company": "Acme Corp",
  "role": "Software Engineer",
  "status": "Applied",
  "appliedDate": "2025-03-01",
  "followUpDate": "2025-03-10",
  "notes": "Applied through LinkedIn",
  "jobLink": "https://acme.com/jobs/123"
}
```

Field	Required	Notes
company	Yes	
role	Yes	
status	No	Defaults to "Applied"
appliedDate	No	Defaults to current date

Field	Required	Notes
`followUpDate`	No	When set, the cron job will send a reminder on this date
`notes`	No	
`jobLink`	No	

Responses:

Status	Body	Condition
`201`	Created application object	Success
`401`	`{ "message": "Not authorized" }`	Missing or invalid token
`500`	`{ "message": "Server error" }`	Unexpected server failure

`PUT /api/applications/:id`

Updates an existing application. Only the owning user may update their own applications.

Middleware chain: `protect` → `validateObjectId` → `updateApplication`

Request body: Any subset of application fields to update.

Responses:

Status	Body	Condition
`200`	Updated application object	Success
`400`	`{ "message": "Invalid ID" }`	`id` is not a valid ObjectId
`401`	`{ "message": "Not authorized" }`	Token invalid or user does not own this application
`404`	`{ "message": "Application not found" }`	No document with that `id`
`500`	`{ "message": "Server error" }`	Unexpected server failure

`DELETE /api/applications/:id`

Permanently deletes an application. Only the owning user may delete their own applications.

Middleware chain: `protect` → `validateObjectId` → `deleteApplication`

Responses:

Status	Body	Condition
`200`	`{ "message": "Application removed" }`	Deleted successfully

Status	Body	Condition
400	{ "message": "Invalid ID" }	id is not a valid ObjectId
401	{ "message": "Not authorized" }	Token invalid or user does not own this application
404	{ "message": "Application not found" }	No document with that id
500	{ "message": "Server error" }	Unexpected server failure

9) Background Jobs

File: server/jobs/reminder.job.js

The reminder job is the only background task in the system. It is registered at server startup by server.js via require('./jobs/reminder.job') and runs inside the main Node.js process using node-cron.

9.1 sendReminders

Schedule: 0 9 * * * — every day at 9:00 AM server local time.

Logic:

Step	Description
1	Construct today as a Date with time set to 00:00:00.000
2	Query Application.find({ followUpDate: { $lte: today }, reminderSent: false }) with .populate('user') to get the user's email
3	For each matching application: send a reminder email via Nodemailer, then set reminderSent = true and save the document
4	Log the count of emails sent

Email format:

Field	Value
From	process.env.EMAIL_USER
To	application.user.email
Subject	Follow-up Reminder: <company>
Body	Plain text: Reminder to follow up on your <role> application at <company>.

Idempotency: The reminderSent flag ensures each application triggers at most one reminder email, even if the job runs multiple times in a day or if the server restarts.

---

10) Email System

File: server/config/mailer.js

10.1 Transport

A single shared Nodemailer transporter is created at module load time and exported for use by the reminder job.

```
nodemailer.createTransport({
    service: 'gmail',
    auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS }
})
```

The transporter uses Gmail's built-in SMTP settings. `EMAIL_PASS` must be a **Gmail App Password**, not the account's login password.

10.2 Email Types

Trigger	Subject	Content
Follow-up date reached (cron)	`Follow-up Reminder: <company>`	Plain text reminder to follow up on the specific application

11) Configuration and Environment

Required Variables

Variable	Description
`MONGO_URI`	MongoDB Atlas connection string
`JWT_SECRET`	Secret key used to sign and verify JWT tokens (use a long, random string)
`EMAIL_USER`	Gmail address used as the SMTP sender account
`EMAIL_PASS`	Gmail App Password for the sender account (not the account login password)
`CORS_ORIGIN`	Comma-separated list of allowed frontend origins (e.g. `http://localhost:5173`)
`PORT`	Port the Express server listens on (defaults to `5000` if not set)
`NODE_ENV`	Set to `production` to enable secure cookies and strict `sameSite` policy

Development Server Commands

```
# Install dependencies
npm install

# Start server with hot-reload (nodemon)
npm run dev
```

```
# Start server (production)
npm start
```

Security Notes for Production

- Set `NODE_ENV=production` to enable `secure: true` and `sameSite: 'strict'` on the auth cookie.
- If the frontend and backend are deployed on **different domains**, update the cookie options to `sameSite: 'none'` and `secure: true`, and ensure `CORS_ORIGIN` includes the exact frontend domain.
- Use a strong, randomly generated `JWT_SECRET` (minimum 32 characters). Never commit it to source control.
- Gmail App Passwords require 2-Factor Authentication to be enabled on the sender Google account.

---

12) Component-by-Component Reference

`server/server.js`

Entry point. Calls `dotenv.config()` first to ensure environment variables are available before any other module loads. Connects to MongoDB, registers the cron job by requiring `reminder.job.js`, then starts the HTTP server on `PORT`.

`server/app.js`

Configures and exports the Express application. Applies global middleware (Helmet, CORS, cookie-parser, JSON body parser), attaches the rate limiter to the auth router, and mounts the two route modules at `/api/auth` and `/api/applications`.

`server/config/db.js`

Exports `connectDB`, an async function that calls `mongoose.connect()` with the `MONGO_URI` environment variable. Logs success or exits the process with code `1` on connection failure, preventing the server from running without a database.

`server/config/mailer.js`

Creates and exports a single shared Nodemailer transporter configured for Gmail SMTP. Instantiated once at module load; imported by the reminder job.

`server/models/User.js`

Defines the `User` Mongoose schema. Email is enforced as unique and lowercased. Password is stored as a bcrypt hash — the schema itself applies no transformation; hashing is done in the controller before `User.create()`.

`server/models/Application.js`

Defines the `Application` Mongoose schema. The `user` field holds the ObjectId of the owning user, enforcing data isolation. The `status` field is constrained to four enum values. The `reminderSent` flag controls the cron job's idempotency. Includes `timestamps: true` for automatic `createdAt` / `updatedAt` fields.

server/middleware/auth.middleware.js

Exports the `protect` middleware. Checks the `Authorization` header first, then the `token` cookie. Verifies the JWT with `jwt.verify()`, loads the user from MongoDB, and attaches the user (without password) to `req.user`. Rejects with `401` on any failure.

server/middleware/validateObjectId.js

Exports `validateObjectId`. A lightweight guard that calls `mongoose.Types.ObjectId.isValid(req.params.id)` and returns `400 Invalid ID` on failure, keeping controller code free of ObjectId casting concerns.

server/controllers/auth.controller.js

Implements `register`, `login`, and `logout`. Defines the `generateToken` helper (signs a JWT with `userId` payload) and the `setAuthCookie` helper (sets cookie with environment-aware security flags). Both helpers are private to this module.

server/controllers/application.controller.js

Implements the four CRUD handlers. All handlers scope queries to `req.user._id` provided by the auth middleware. `updateApplication` and `deleteApplication` perform an explicit ownership check after fetching the document, returning `401` if the document's `user` field does not match the requester's ID.

server/routes/auth.routes.js

Mounts `register`, `login`, and `logout` on three `POST` routes. No auth middleware — these endpoints must be publicly accessible.

server/routes/application.routes.js

Applies `protect` at the router level via `router.use(protect)`, so every route in this file is automatically authenticated. `PUT` and `DELETE` also chain `validateObjectId` before the controller.

server/jobs/reminder.job.js

Exports `sendReminders` and immediately schedules it with `cron.schedule('0 9 * * *', sendReminders)`. The job queries for all applications with an overdue, unsent follow-up date, sends each user a plain-text reminder email, and marks each application as `reminderSent: true` to prevent re-sending.